Honeypot IDS – Pool test report

Generated: 2026-10-07 17:55:03
Target: ids (local) (https://localhost)
Exploits: A: CVE-2024-2387, B: CVE-2025-47577, C: GENERIC-WP-003, D: CVE-2024-27956, E: CVE-2022-0739, A: CVE-2024-1071, B: CVE-2025-4403, C: GENERIC-WP-002, D: CVE-2024-1071, E: CVE-2025-4403
Windows: 5

Result

SHARED EXPECTED **Pools are shared, as expected under resource pressure**

- Pool 2 is shared by Attacker A, Attacker D.
- Pool 3 is shared by Attacker B, Attacker E.
- This is the designed fallback: pool growth is capped (low memory: 1058 MB available, need 2560 MB), so further attackers were assigned to the ready pools round-robin.
- Pool 2 (owned by window D) also served window(s) A.
- Pool 3 (owned by window E) also served window(s) B.

What is being tested

Attackers are told apart by session, not by address. Each test window has its own cookie jar and its own User-Agent and language, so the proxy sees 5 separate attackers. By default every malicious session is given a honeypot pool **of its own** (an eshop container with its own seeded database): the router hands a new attacker a free ready pool no other session owns, and the pools scale with the attacker sessions – a session that finds no free pool borrows one round-robin while pool_manager builds one for it, then moves there. Scaling stops at the configured resource limit (POOL_MAX_MEMORY_MB / POOL_MAX_CPUS / POOL_MAX) or when the host is short of memory, CPU or disk; from then on further attackers share the ready pools in strict round-robin order.

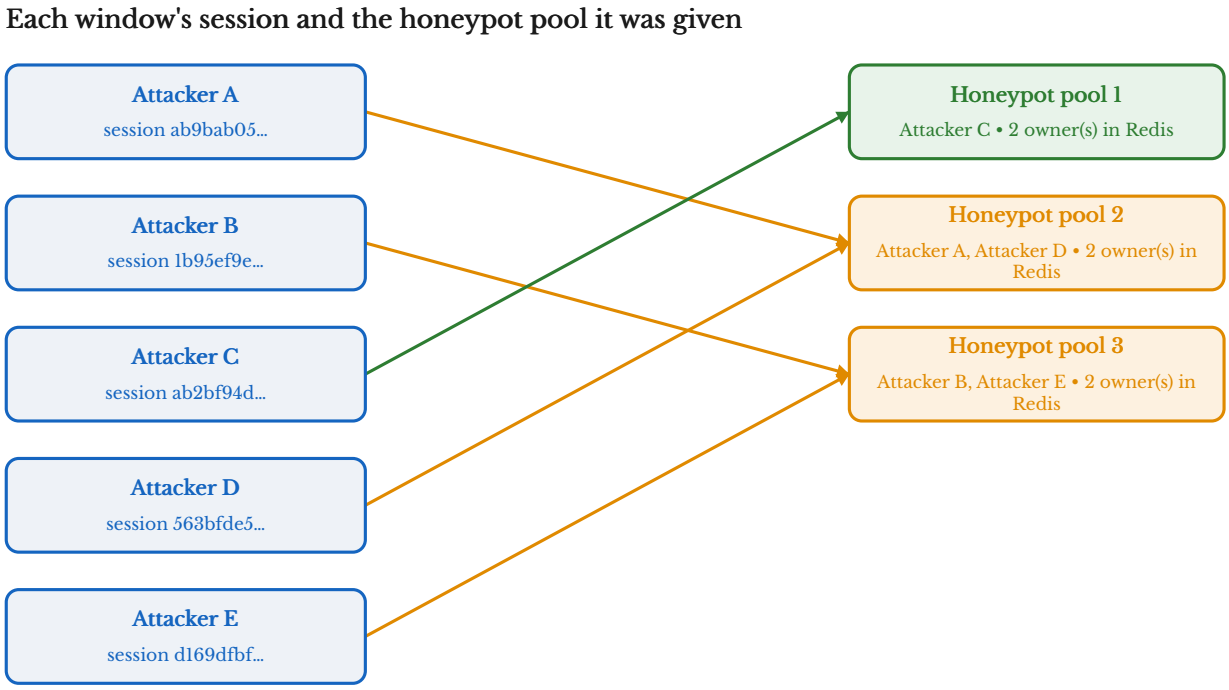


Figure 1. One arrow per test window, from its session to the honeypot pool the router bound it to. Green: the pool belongs to that window alone. Orange: shared under resource pressure (designed). Red: shared or merged without a reason. Grey: not bound.

Windows

Window	Session	Honeypot pool	Language	User-Agent (tail)
Attacker A	ab9bab05…	pool 2	en-US,en;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/A
Attacker B	1b95ef9e…	pool 3	de-DE,de;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/B
Attacker C	ab2bf94d…	pool 1	fr-FR,fr;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/C
Attacker D	563bfde5…	pool 2	es-ES,es;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/D
Attacker E	d169dfbf…	pool 3	it-IT,it;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/E

Scaling and latency

pool_manager adds one honeypot pool (an eshop container plus its own database seeded from production) for every attacker session that finds no free pool, up to the resource limit. These figures come from pool_manager's own event log (build phases, hand-overs with their wait time, resource-limit changes) and from the windows themselves (routing read after every step and every 2 s while waiting; the duration of each page load).

Windows	5, of which 2 attacked only after pools were pre-built for them
Run time	7:42
Resource limit	unknown (pool_manager status not readable)

Pools added during the run	0 built
Round-robin while not scaled yet (case A)	0 window(s) borrowed a pool
Round-robin at the resource limit (case B)	2 window(s) share a pool for good
Forced scale-down at the end	3 → 3 pool(s) (not finished after 5 min)
Pre-build for the delayed windows	0 of 2 ready after 5 s (growth capped: low memory: 1055 MB available, need 2560 MB)

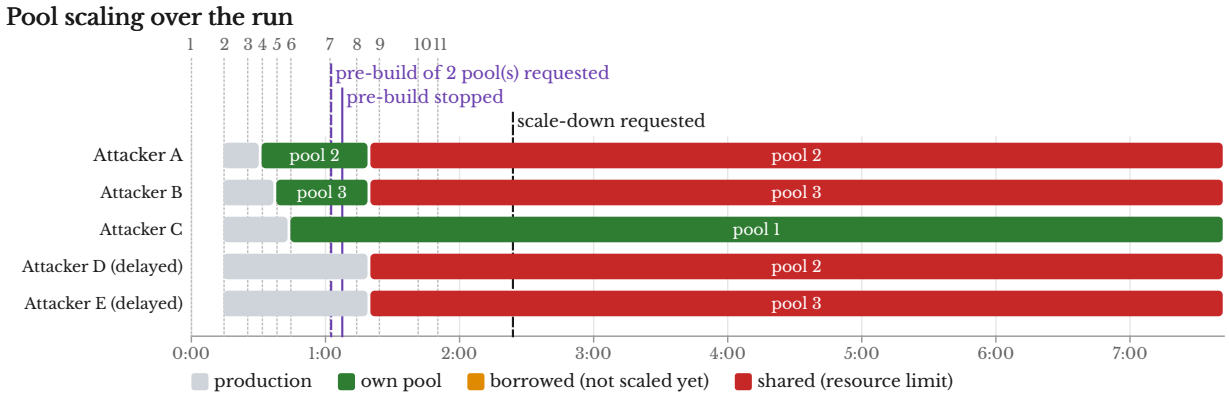


Figure 2. Where each window was served over the run (grey: production, green: its own pool, orange: a borrowed pool while its own was being built, red: shared because the resource limit was reached), and every pool built during the run, its phases shaded dark to light. Dashed grey arrows: a finished pool handed to the window that waited for it. Numbered dotted lines: the steps below.

No pool was built during the run: the ready pools covered every window, or growth was capped from the start.

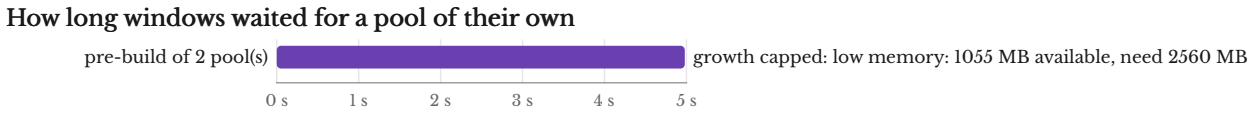


Figure 3. Orange: time a window spent on a borrowed pool (assigned round-robin because no pool was free yet) until it was moved to the pool built for it, as seen by the window (2 s resolution); pool_manager's own measurement in brackets. Red: the window was left on the borrowed pool because the resource limit was reached. Purple: time to pre-build the pools for the delayed windows.

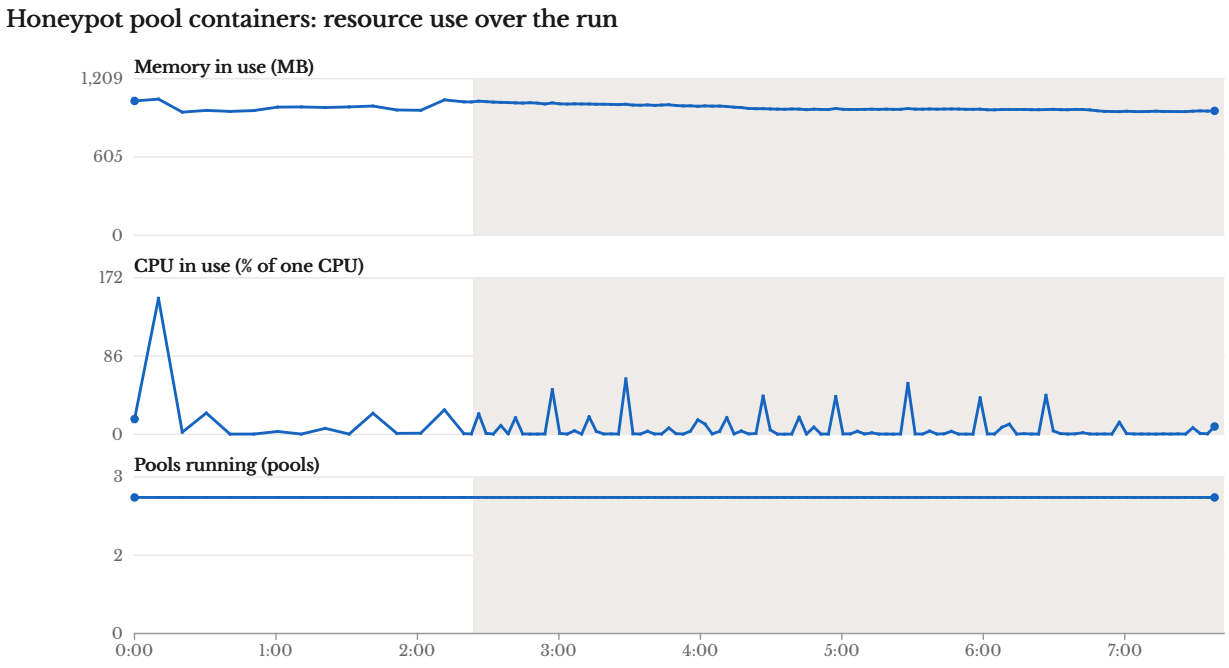


Figure 4. Memory, CPU and number of running honeypot pools (eshop + database containers, docker stats every 10 s, every 3 s around the scale-down). Dashed lines: green a pool finished building, red the resource limit was reached, brown a pool was removed. Shaded: the forced scale-down at the end of the run.

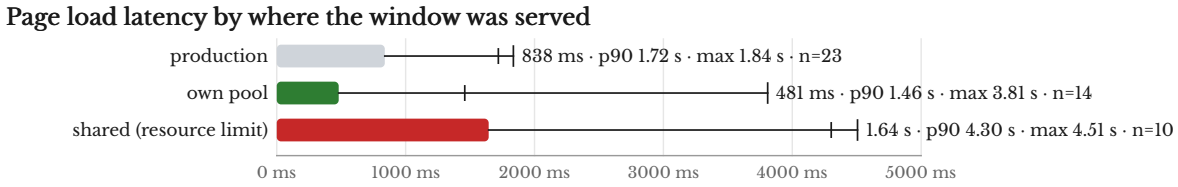


Figure 5. Every page load of every window during the run (browser navigation start to load finished), grouped by where that window was routed right after it. Bar: median; short tick: 90th percentile; line end: slowest load.

Round-robin assignment

A diverted session is only ever put on a pool another session uses in two situations, both handled round-robin over the ready pools:

A. Not scaled up yet (no free pool, more can be built)

Not observed: every window found a free pool the moment it was diverted. The delayed windows waited for pre-built pools, which is how this case is avoided.

B. Resource limit reached (no more pools can be built)

Window	Pool	Since	Shares with	How
Attacker D	pool 2	1:19	Attacker A	assigned while capped
Attacker E	pool 3	1:19	Attacker B	assigned while capped

Scale-down

The idle timeout is unknown (pool_manager status not readable). Waiting that long is not practical in a test, so the run ends by forcing it: the windows stop sending requests, their sessions are released, and pool_manager removes the pools that are no longer needed at once. Other sessions' pools are not touched.

Requested at	2:24
Sessions released	*5*
Finished	not within *5* min
Pools	3 → 3 (1, 2, 3 → 1, 2, 3)
Pools removed	none (the compose-declared pools cover the spare count; nothing above it was running)
Memory in use (measured)	1,029 MB → 961 MB (-69 MB)
CPU in use (measured)	1 % → 9 %
Containers running	6 → 6

Cart persistence

A visitor diverted to a honeypot is served by a different database, so WooCommerce's own cart session does not exist there. The storefront therefore carries the cart in a cookie and rebuilds it on whichever instance serves the next request. Each window filled its cart on production; the cart was then read back after every step, through the same cookies and routing as the window's own traffic.

Step	Attacker A	Attacker B	Attacker C	Attacker D	Attacker E
All windows open the shop	no cart yet on production	no cart yet on production	no cart yet on production	no cart yet on production	no cart yet on production
Every window fills its cart	same cart on production	same cart on production	same cart on production	same cart on production	same cart on production
Attacker A attacks with CVE-2024-2387	same cart on pool 2	same cart on production	same cart on production	same cart on production	same cart on production
Attacker B attacks with CVE-2025-47577	same cart on pool 2	same cart on pool 3	same cart on production	same cart on production	same cart on production
Attacker C attacks with GENERIC-WP-003	same cart on pool 2	same cart on pool 3	same cart on pool 1	same cart on production	same cart on production
Honeypot pools scale up	same cart on pool 2	same cart on pool 3	same cart on pool 1	same cart on production	same cart on production
Pools pre-built for 2 delayed window(s)	same cart on pool 2	same cart on pool 3	same cart on pool 1	same cart on production	same cart on production
Delayed windows D, E attack together	same cart on pool 2	same cart on pool 3	same cart on pool 1	same cart on pool 2	same cart on pool 3
Honeypot pools scale up	same cart on pool 2	same cart on pool 3	same cart on pool 1	same cart on pool 2	same cart on pool 3
All windows run a second exploit	same cart on pool 2	same cart on pool 3	same cart on pool 1	same cart on pool 2	same cart on pool 3

Every window opens its cart	same cart on pool 2	same cart on pool 3	same cart on pool 1	same cart on pool 2	same cart on pool 3

Every cart stayed exactly as filled, on production and in every honeypot pool.

Exploits run per window

Each window runs a different exploit, picked from the ones a live probe showed divert within three attempts. If a chosen exploit still did not divert in three attempts, the next candidate is used instead.

Window	Exploit	Name	Routing	Pool
A	CVE-2024-2387	Advanced Form Integration SQL injection	diverted after 1	pool 2
B	CVE-2025-47577	Gift Voucher file upload RCE	diverted after 1	pool 3
C	GENERIC-WP-003	User enumeration via ?author=	diverted after 1	pool 1
D	CVE-2024-27956	WP Automatic SQL injection	diverted after 1	pool 2
E	CVE-2022-0739	BookingPress SQL injection	diverted after 1	pool 3
A	CVE-2024-1071	Ultimate Member SQL injection	diverted after 1	pool 2
B	CVE-2025-4403	Drag & Drop file upload MIME-type bypass	diverted after 1	pool 3
C	GENERIC-WP-002	Installer file access	diverted after 1	pool 1
D	CVE-2024-1071	Ultimate Member SQL injection	diverted after 1	pool 2
E	CVE-2025-4403	Drag & Drop file upload MIME-type bypass	diverted after 1	pool 3

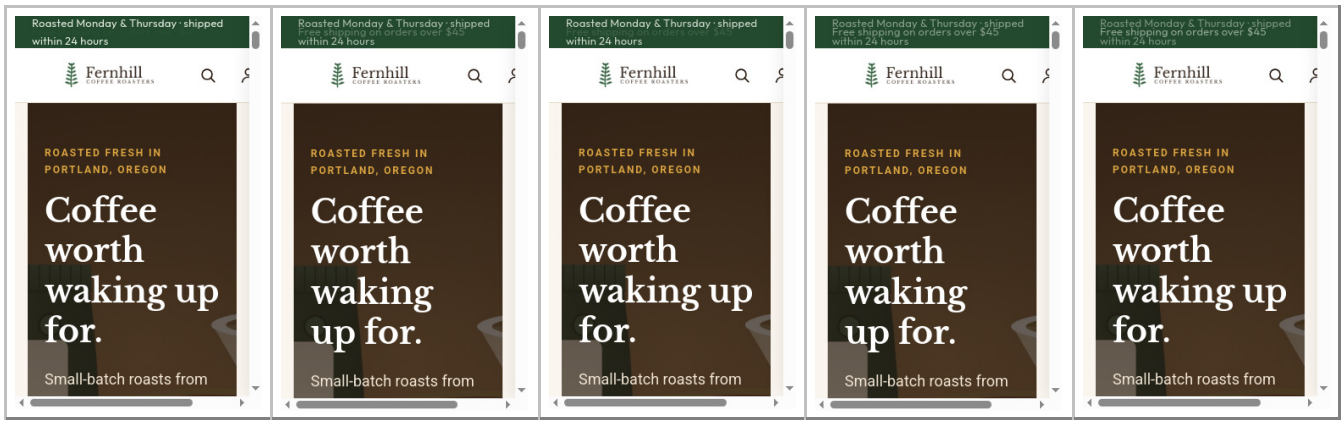
Step by step

All windows move in every step. An attacker's pool must appear in their own step, differ from the earlier ones, and never change afterwards – except once, from a pool it only borrowed while its own was being built to that new pool.

Step 1: All windows open the shop

5 new sessions, all routed to production.

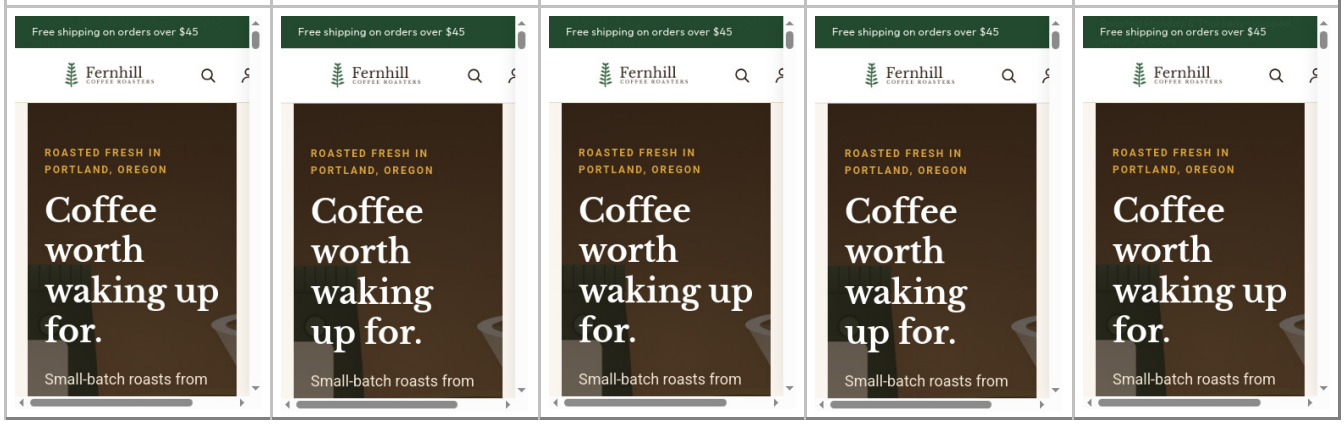
Attacker A	Attacker B	Attacker C	Attacker D	Attacker E
Browses the shop	Browses the shop	Browses the shop	Browses the shop	Browses the shop
PRODUCTION no pool • score 0	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0
cart empty	cart empty	cart empty	cart empty	cart empty



Step 2: Every window fills its cart

Each window adds its own products. These carts are the baseline: they must look exactly the same after the session is routed to a honeypot pool.

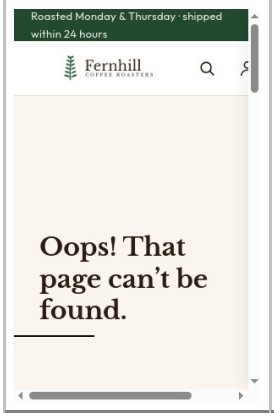
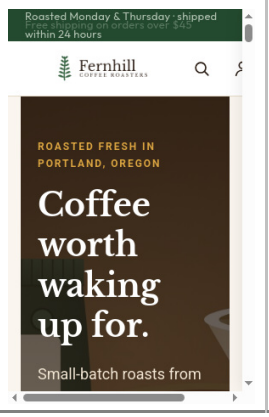
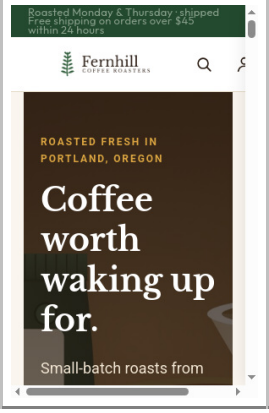
Attacker A	Attacker B	Attacker C	Attacker D	Attacker E
Adds ceramic-pour-over-dripper × 1, paper-filters-size-02 × 2	Adds hand-burr-coffee-grinder × 1, stainless-milk-pitcher × 1	Adds enamel-camp-mug × 2, fernhill-canvas-tote × 1	Adds gooseneck-electric-kettle × 1, logo-cap × 2	Adds digital-brew-scale × 1
PRODUCTION no pool • score 0	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1



Step 3: Attacker A attacks with CVE-2024-2387

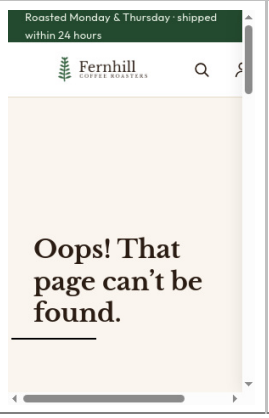
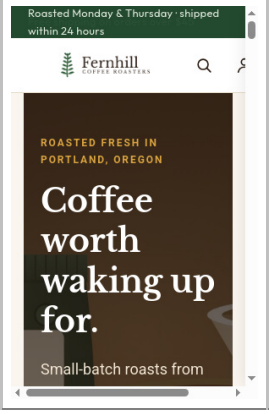
Advanced Form Integration SQL injection. The session must be bound to a pool no other window has -- or, when no pool is free yet, borrow one round-robin until the pool being built for it is ready -- and the windows that have not attacked yet must stay on production.

Attacker A	Attacker B	Attacker C	Attacker D	Attacker E
Opens CVE-2024-2387 (1 attempt(s))	Browses the shop	Browses the shop	Browses the shop	Browses the shop
HONEYPOT pool 2 • score 100	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0
$46.00	**$113.00**	**$60.00**	**$121.00**	**$49.00**

Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	Digital Brew Scale with Timer × 1
				

Step 4: Attacker B attacks with CVE-2025-47577

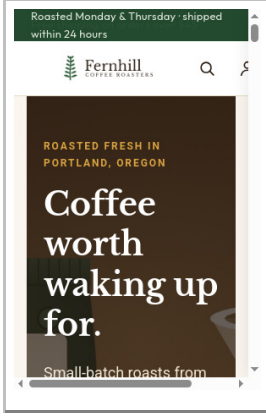
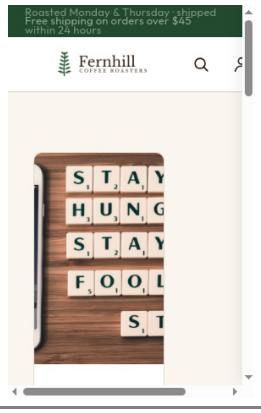
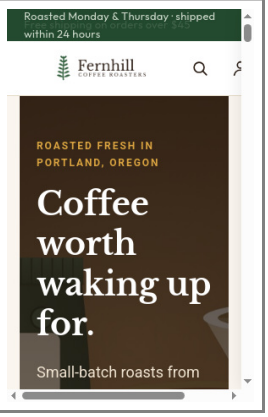
Gift Voucher file upload RCE. The session must be bound to a pool no other window has -- or, when no pool is free yet, borrow one round-robin until the pool being built for it is ready -- and the windows that have not attacked yet must stay on production.

Attacker A	Attacker B	Attacker C	Attacker D	Attacker E
Browses the shop	Opens CVE-2025-47577 (1 attempt(s))	Browses the shop	Browses the shop	Browses the shop
HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 95	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

Step 5: Attacker C attacks with GENERIC-WP-003

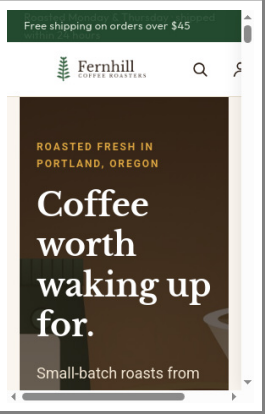
User enumeration via ?author=. The session must be bound to a pool no other window has -- or, when no pool is free yet, borrow one round-robin until the pool being built for it is ready -- and the windows that have not attacked yet must stay on production.

Attacker A	Attacker B	Attacker C	Attacker D	Attacker E
Browses the shop	Browses the shop	Opens GENERIC-WP-003 (1 attempt(s))	Browses the shop	Browses the shop
HONEYPOT	HONEYPOT	HONEYPOT	PRODUCTION	PRODUCTION

pool 2 • score 100	pool 3 • score 100	pool 1 • score 100	no pool • score 0	no pool • score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

Step 6: Honeypot pools scale up

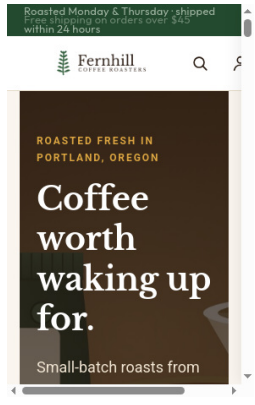
Windows that found no free pool (not scaled up yet) borrowed a ready one round-robin while pool_manager built a pool (eshop + seeded database) for each of them, and must now each be in their own. Once the resource limit is reached no more pools are built and the remaining windows keep sharing the ready pools round-robin.

Attacker A	Attacker B	Attacker C	Attacker D	Attacker E
Browses the shop	Browses the shop	Browses the shop	Browses the shop	Browses the shop
HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 100	HONEYPOT pool 1 • score 100	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

Step 7: Pools pre-built for 2 delayed window(s)

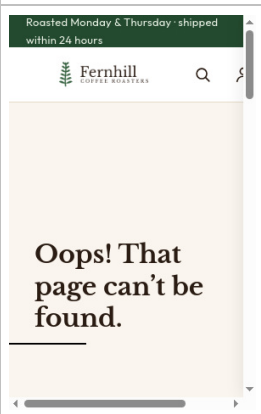
Before windows D, E attack, the test asks pool_manager for 2 free pool(s) and waits until they are ready, so these windows find a pool of their own the moment they are diverted instead of borrowing one round-robin. If the resource limit stops the build first, they share the ready pools round-robin instead.

Attacker A	Attacker B	Attacker C	Attacker D	Attacker E
Browses the shop	Browses the shop	Browses the shop	Waits for its pool:	Waits for its pool:

			only 0 of 2 free pool(s) after 5 s (growth capped: low memory: 1055 MB available, need 2560 MB)	only 0 of 2 free pool(s) after 5 s (growth capped: low memory: 1055 MB available, need 2560 MB)
HONEYPOT pool 2 • score 100	**HONEYPOT** pool 3 • score 100	**HONEYPOT** pool 1 • score 100	**PRODUCTION** no pool • score 0	**PRODUCTION** no pool • score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

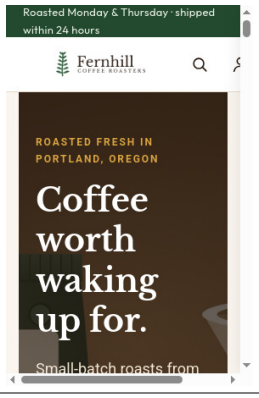
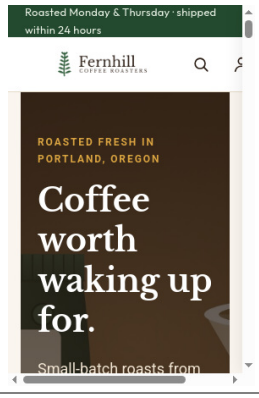
Step 8: Delayed windows D, E attack together

These windows attack only now that pools were built for them: each must get a free pool of its own at once (no borrowing) -- unless the resource limit stopped the build, in which case they share the ready pools round-robin.

Attacker A	Attacker B	Attacker C	Attacker D	Attacker E
Browses the shop	Browses the shop	Browses the shop	Opens CVE-2024-27956 (1 attempt(s))	Opens CVE-2022-0739 (1 attempt(s))
HONEYPOT pool 2 • score 100	**HONEYPOT** pool 3 • score 100	**HONEYPOT** pool 1 • score 100	**HONEYPOT** pool 2 • score 100	**HONEYPOT** pool 3 • score 100
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				0

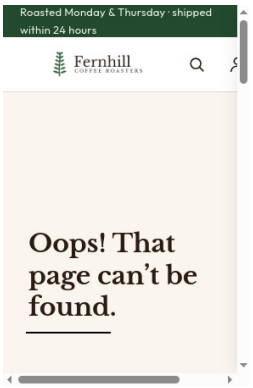
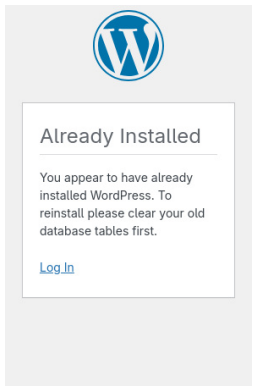
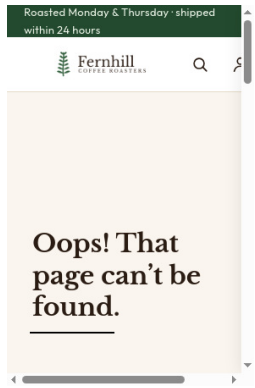
Step 9: Honeypot pools scale up

Windows that found no free pool (not scaled up yet) borrowed a ready one round-robin while pool_manager built a pool (eshop + seeded database) for each of them, and must now each be in their own. Once the resource limit is reached no more pools are built and the remaining windows keep sharing the ready pools round-robin.

Attacker A	Attacker B	Attacker C	Attacker D	Attacker E
Browses the shop	Browses the shop	Browses the shop	Browses the shop	Browses the shop
HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 100	HONEYPOT pool 1 • score 100	HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 100
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

Step 10: All windows run a second exploit

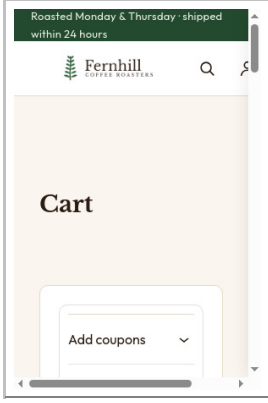
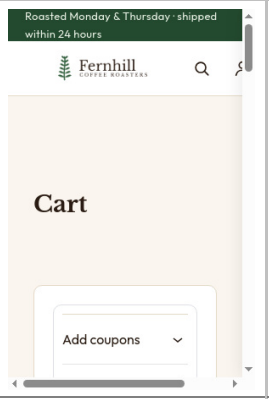
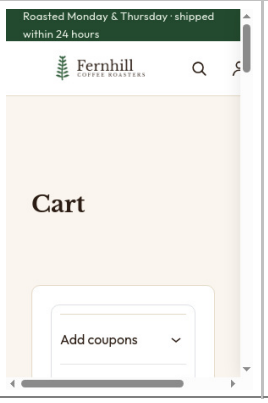
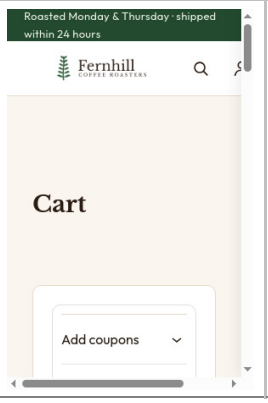
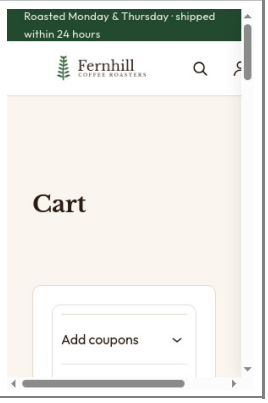
Sticky binding: each second exploit must land in the pool the window already owns, nobody changes pool, nobody is merged.

Attacker A	Attacker B	Attacker C	Attacker D	Attacker E
Opens CVE-2024-1071	Opens CVE-2025-4403	Opens GENERIC-WP-002	Opens CVE-2024-1071	Opens CVE-2025-4403
HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 100	HONEYPOT pool 1 • score 100	HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 100
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
0			0	

Step 11: Every window opens its cart

The cart page, served by each window's honeypot pool, must show the same products

and quantities that were added on production.

Attacker A	Attacker B	Attacker C	Attacker D	Attacker E
Opens the cart page	Opens the cart page	Opens the cart page	Opens the cart page	Opens the cart page
HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 100	HONEYPOT pool 1 • score 100	HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 100
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1	**$121.00** Gooseneck Electric Kettle 0.9 L × 1 Fernhill Logo Cap × 2	**$49.00** Digital Brew Scale with Timer × 1
				

What each honeypot container saw and changed

Read from each pool's own eshop and database containers since the run started (2026-10-07T15:47:18Z). Requests are matched to windows by their User-Agent tag, so a pool should show its owner's exploits and nobody else's. A pool no window owns is shown for contrast.

pool 1 window C

Windows seen	C (15 requests)
Requests received	• C GET /?author=1 → 301 • C GET /author/admin/ → 200 • C GET /wp-json/wc/store/v1/cart → 200 • C GET / → 200 • C GET /wp-admin/install.php → 200 • C GET /cart/ → 200
Files changed	• no files written
Database activity	questions +13411, select +12670, insert +34, update +10, delete +18

pool 2 window D

Windows seen	A, D (26 requests)
Requests received	• A GET /?page=advanced-form-integration-log&integration_id=1%27%20UNION%20SELECT%20user_login%2Cu → 404 • A GET /wp-json/wc/store/v1/cart → 200 • A GET / → 200 • D GET /wp-content/plugins/wp-automatic/inc/csv.php?q=1%27%20UNION%20SELECT%20user_login%2Cuser_p → 404 • D GET /wp-json/wc/store/v1/cart → 200

	• D GET / → 200 • A GET /wp-admin/admin-ajax.php?action=um_get_members&directory_id=1%27%20UNION%20SELECT%20user_l → 400 • D GET /wp-admin/admin-ajax.php?action=um_get_members&directory_id=1%27%20UNION%20SELECT%20user_l → 400
Files changed	• no files written
Database activity	questions +14574, select +13784, insert +34, update +14, delete +17

pool 3 window E

Windows seen	B, E (24 requests)
Requests received	• B GET /?action=mwb_wgm_preview_mail&attachment=shell.php → 404 • B GET /wp-json/wc/store/v1/cart → 200 • B GET / → 200 • E GET /wp-admin/admin-ajax.php?action=bookingpress_front_get_category_services&total_service=1%2 → 400 • E GET /wp-json/wc/store/v1/cart → 200 • E GET / → 200 • B GET /?action=dnd-wc-upload-file&name=shell.php → 404 • E GET /?action=dnd-wc-upload-file&name=shell.php → 404
Files changed	• no files written
Database activity	questions +5718, select +5421, insert +33, update +11, delete +16

Pool registry at export time

Growth capped – low memory: 1058 MB available, need 2560 MB. New attackers reuse the existing pools round-robin instead of getting a new one.
Ready pools: 3 (1, 2, 3). Free (unowned) spares: 0. Attackers per pool: pool 1: 2, pool 2: 2, pool 3: 2. Round-robin reuse assignments so far: 3.